

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ

Заместитель директора
по учебной работе

_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 14

ПРОГРАММИРОВАНИЕ КЛАССОВ. ПЕРЕГРУЗКА ОПЕРАТОРОВ

ЛАБОРАТОРНАЯ РАБОТА № 14.1

ПРОГРАММИРОВАНИЕ КЛАССОВ. ПЕРЕГРУЗКА АРИФМЕТИЧЕСКИХ
И ЛОГИЧЕСКИХ ОПЕРАТОРОВ

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № _____ от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Программирование классов. Перегрузка арифметических и логических операторов

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Специальные методы

Классы поддерживают следующие специальные методы:

– **__call__()** - позволяет обработать вызов экземпляра класса как вызов функции. **Формат метода:**

```
__call__ (self [, <Параметр1>[, ..., <ПараметрN>]])
```

Пример:

```
class MyClass:
```

```
    def __init__(self, m):
```

```
        self.msg = m
```

```
    def __call__(self):
```

```
        print(self.msg)
```

```
c1 = MyClass("Значение1") # Создание экземпляра класса
```

```
c2 = MyClass("Значение2") # Создание экземпляра класса
```

```
c1()           # Выведет: Значение1
```

```
c2()           # Выведет: Значение2
```

Результат работы приложения:

Значение1

Значение2

– **__getattr__(self, <Атрибут>)** - вызывается при обращении к несуществующему атрибуту класса. **Пример:**

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.i = 20
```

```
    def __getattr__(self, attr):
```

```
        print("Вызван метод __getattr__()")
```

```

    return 0
c = MyClass()
# Атрибут i существует
print(c.i) # Выведет: 20. Метод __getattr__()
# Атрибут s не существует
print(c.s) # Выведет: Вызван метод __getattr__()

```

Результат работы приложения:

```

20
Вызван метод __getattr__()
0

```

- **__getattr__**(self, <Атрибут>) - вызывается при обращении к любому атрибуту класса. Необходимо учитывать, что использование точечной нотации (для обращения к атрибуту класса) внутри этого метода приведет к заикливанию. Чтобы избежать заикливания, следует вызывать метод **__getattr__**() объекта **object**. Внутри метода нужно вернуть значение атрибута или возбудить исключение **AttributeError**. **Пример:**

```

class MyClass:
    def __init__(self):
        self.i = 20
    def __getattr__(self, attr):
        print("Вызван метод __getattr__()")
        return object.__getattr__(self, attr) # Только так!!!
c = MyClass()
print(c.i) # Выведет: Вызван метод __getattr__() 20

```

Результат работы приложения:

```

Вызван метод __getattr__()
20

```

- **__setattr__**(self, <Атрибут>, <Значение>) - вызывается при попытке присваивания значения атрибуту экземпляра класса. Если внутри метода необходимо присвоить значение атрибуту, то следует использовать словарь **__dict__**, иначе при точечной нотации метод **__setattr__**() будет вызван повторно, что приведет к заикливанию. **Пример:**

```

class MyClass:
    def __setattr__(self, attr, value):
        print("Вызван метод __setattr__()")
        self.__dict__[attr] = value
c = MyClass()
c.i = 10 # Выведет: Вызван метод __setattr__()
print(c.i) # Выведет: 10

```

Результат работы приложения:

```

Вызван метод __setattr__()
10

```

- **__delattr__**(self, <Атрибут>) - вызывается при удалении атрибута с помощью инструкции **del** <Экземпляр класса>.<Атрибут>;

– **__len__(self)** - вызывается при использовании функции **len()**, а также для проверки объекта на логическое значение при отсутствии метода **__bool__()**. Метод должен возвращать положительное целое число. Пример:

```
class MyClass:
    def __len__(self):
        return 50
c = MyClass()
print(len(c)) # Выведет: 50
```

Результат работы приложения:

50

– **__bool__(self)** - вызывается при использовании функции **bool()**;
 – **__int__(self)** - вызывается при преобразовании объекта в целое число с помощью функции **int()**;
 – **__float__(self)** - вызывается при преобразовании объекта в вещественное число с помощью функции **float()**;
 – **__complex__(self)** - вызывается при преобразовании объекта в комплексное число с помощью функции **complex()**;
 – **__round__(self, n)** - вызывается при использовании функции **round()**;
 – **__index__(self)** - вызывается при использовании функций **bin()**, **hex()** и **oct()**;
 – **__repr__(self)** и **__str__(self)** - служат для преобразования объекта в строку. Метод **__repr__()** вызывается при выводе в интерактивной оболочке, а также при использовании функции **repr()**. Метод **__str__()** вызывается при выводе с помощью функции **print()**, а также при использовании функции **str()**. Если метод **__str__()** отсутствует, то будет вызван метод **__repr__()**. В качестве значения методы **__repr__()** и **__str__()** должны возвращать строку. **Пример:**

```
class MyClass:
    def __init__(self, m):
        self.msg = m
    def __repr__(self):
        return "Вызван метод __repr__() {0}".format(self.msg)
    def __str__(self):
        return "Вызван метод __str__() {0}".format(self.msg)
c = MyClass("Значение")
print(repr(c)) # Выведет: Вызван метод __repr__() Значение
print(str(c)) # Выведет: Вызван метод __str__() Значение
print(c)      # Выведет: Вызван метод __str__() Значение
```

Результат работы приложения:

Вызван метод __repr__() Значение

Вызван метод __str__() Значение

Вызван метод __str__() Значение

– **__hash__(self)** - этот метод следует переопределить, если экземпляр класса планируется использовать в качестве ключа словаря или внутри множества.

Пример:

```
class MyClass:
```

```
def __init__(self, y):
    self.x = y
def __hash__(self):
    return hash(self.x)
c = MyClass(10)
d = {}
d[c] = "Значение"
print(d[c]) # Выведет: Значение
```

Результат работы приложения:
Значение

Классы поддерживают еще несколько специальных методов, которые применяются лишь в особых случаях и будут рассмотрены позднее.

4.2 Перегрузка операторов

Перегрузка операторов позволяет экземплярам классов участвовать в обычных операциях. Чтобы перегрузить оператор, необходимо в классе определить метод со специальным названием. Для перегрузки математических операторов используются следующие методы:

- $x + y$ - сложение: `x.__add__(y)`;
- $y + x$ - сложение (экземпляр класса справа): `x.__radd__(y)`;
- $x += y$ - сложение и присваивание: `x.__iadd__(y)`;
- $x - y$ - вычитание: `x.__sub__(y)`;
- $y - x$ - вычитание (экземпляр класса справа): `x.__rsub__(y)`;
- $x -= y$ - вычитание и присваивание: `x.__isub__(y)`;
- $x * y$ - умножение: `x.__mul__(y)`;
- $y * x$ - умножение (экземпляр класса справа): `x.__rmul__(y)`;
- $x *= y$ - умножение и присваивание: `x.__imul__(y)`;
- x / y - деление: `x.__truediv__(y)`;
- y / x - деление (экземпляр класса справа): `x.__rtruediv__(y)`;
- $x /= y$ - деление и присваивание: `x.__itruediv__(y)`;
- $x // y$ - деление с округлением вниз: `x.__floordiv__(y)`;
- $y // x$ - деление с округлением вниз (экземпляр класса справа): `x.__rfloordiv__(y)`;
- $x //= y$ - деление с округлением вниз и присваивание: `x.__ifloordiv__(y)`;
- $x \% y$ - остаток от деления: `x.__mod__(y)`;
- $y \% x$ - остаток от деления (экземпляр класса справа): `x.__mod__(y)`;
- $x \% = y$ - остаток от деления и присваивание: `x.__imod__(y)`;
- $x ** y$ - возведение в степень: `x.__pow__(y)`;
- $y ** x$ - возведение в степень (экземпляр класса справа): `x.__rpow__(y)`;
- $x ** = y$ - возведение в степень и присваивание: `x.__ipow__(y)`;
- $-x$ - унарный минус: `x.__neg__()`;
- $+x$ - унарный плюс: `x.__pos__()`;
- `abs(x)` - абсолютное значение: `x.__abs__()`.

Пример перегрузки математических операторов:

```
class MyClass:
    def __init__(self, y):
        self.x = y
    def __add__(self, y): # Перегрузка оператора +
        print("Экземпляр слева")
        return self.x + y
    def __radd__(self, y): # Перегрузка оператора +
        print("Экземпляр справа")
        return self.x + y
    def __iadd__(self, y): # Перегрузка оператора +=
        print("Сложение с присваиванием")
        self.x += y
        return self
c = MyClass(50)
print(c + 10) # Выведет: Экземпляр слева 60
print(20 + c) # Выведет: Экземпляр справа 70
c += 30 # Выведет: Сложение с присваиванием
print(c.x) # Выведет: 80
```

Результат работы приложения:

Экземпляр слева

60

Экземпляр справа

70

Сложение с присваиванием

80

Методы перегрузки двоичных операторов:

- $\sim x$ - двоичная инверсия: `x.__invert__()`;
- $x \& y$ - двоичное И: `x.__and__(y)`;
- $y \& x$ - двоичное И (экземпляр класса справа): `x.__rand__(y)`;
- $x \&= y$ - двоичное И и присваивание: `x.__iand__(y)`;
- $x | y$ - двоичное ИЛИ: `x.__or__(y)`;
- $y | x$ - двоичное ИЛИ (экземпляр класса справа): `x.__ror__(y)`;
- $x |= y$ - двоичное ИЛИ и присваивание: `x.__ior__(y)`;
- $x \wedge y$ - двоичное исключающее ИЛИ: `x.__xor__(y)`;
- $y \wedge x$ - двоичное исключающее ИЛИ (экземпляр класса справа): `x.__rxor__(y)`;
- $x \wedge= y$ - двоичное исключающее ИЛИ и присваивание: `x.__ixor__(y)`;
- $x \ll y$ - сдвиг влево: `x.__lshift__(y)`;
- $y \ll x$ - сдвиг влево (экземпляр класса справа): `x.__rlshift__(y)`;
- $x \ll= y$ - сдвиг влево и присваивание: `x.__ilshift__(y)`;
- $x \gg y$ - сдвиг вправо: `x.__rshift__(y)`;
- $y \gg x$ - сдвиг вправо (экземпляр класса справа): `x.__rrshift__(y)`;
- $x \gg= y$ - сдвиг вправо и присваивание: `x.__irshift__(y)`.

Перегрузка операторов сравнения производится с помощью следующих методов:

- `x == y` - равно: `x.__eq__(y)`;
- `x != y` - не равно: `x.__ne__(y)`;
- `x < y` - меньше: `x.__lt__(y)`;
- `x > y` - больше: `x.__gt__(y)`;
- `x <= y` - меньше или равно: `x.__le__(y)`;
- `x >= y` - больше или равно: `x.__ge__(y)`;
- `y in x` - проверка на вхождение: `x.__contains__(y)`.

Приведем пример перегрузки операторов сравнения:

```
class MyClass:
```

```
    def __init__(self):
```

```
        self.x = 50
```

```
        self.arr = [1, 2, 3, 4, 5]
```

```
    def __eq__(self, y):      # Перегрузка оператора ==
```

```
        return self.x == y
```

```
    def __contains__(self, y): # Перегрузка оператора in
```

```
        return y in self.arr
```

```
c = MyClass()
```

```
print("Равно" if c == 50 else "Не равно") # Выведет: Равно
```

```
print("Равно" if c == 51 else "Не равно") # Выведет: Не равно
```

```
print("Есть" if 5 in c else "Нет")      # Выведет: Есть
```

Результат работы приложения:

Равно

Не равно

Есть

4.3 Статические методы и методы класса

Внутри класса можно создать метод, который будет доступен без создания экземпляра класса (статический метод). Для этого перед определением метода внутри класса следует указать декоратор **@staticmethod**. Вызов статического метода без создания экземпляра класса осуществляется следующим образом:

```
<Название класса>.<Название метода>(<Параметры>)
```

Кроме того, можно вызвать статический метод через экземпляр класса:

```
<Экземпляр класса>.<Название метода> (<Параметры>)
```

Приведем пример использования статических методов:

```
class MyClass:
```

```
    @staticmethod
```

```
    def func1(x, y):          # Статический метод
```

```
        return x + y
```

```
    def func2(self, x, y):    # Обычный метод в классе
```

```
        return x + y
```

```
    def func3(self, x, y):
```



```

    return MyClass.func1(x, y) # Вызов из метода класса
print(MyClass.func1(10, 20)) # Вызываем статический метод
c = MyClass()
print(c.func2(15, 6))      # Вызываем метод класса
print(c.func1(50, 12))     # Вызываем статический метод
                           # через экземпляр класса
print(c.func3(23, 5))     # Вызываем статический метод
                           # внутри класса

```

Результат работы приложения:

```

30
21
62
28

```

Обратите внимание на то, что в определении статического метода нет параметра **self**. Это означает, что внутри статического метода нет доступа к атрибутам и методам экземпляра класса.

Методы класса создаются с помощью декоратора **@classmethod**. В качестве первого параметра в метод класса передается ссылка на класс, а не на экземпляр класса. Вызов метода класса осуществляется следующим образом:

<Название класса>.<Название метода>(<Параметры>)

Кроме того, можно вызвать метод класса через экземпляр класса:

<Экземпляр класса>.<Название метода> (<Параметры>)

Пример использования методов класса:

```

class MyClass:
    @classmethod
    def func(cls, x): # Метод класса
        print (cls, x)
MyClass.func(10)    # Вызываем метод через название класса
c = MyClass()
c.func(50)          # Вызываем метод класса через экземпляр

```

Результат работы приложения:

```

<class '__main__.MyClass'> 10
<class '__main__.MyClass'> 50

```

4.4 Абстрактные методы

Абстрактные методы содержат только определение метода без реализации. Предполагается, что класс-потомок должен переопределить метод и реализовать его функциональность. Чтобы такое предположение сделать более очевидным, часто внутри абстрактного метода возбуждают исключение:

```

class Class1:
    def func(self, x): # Абстрактный метод
        # Возбуждаем исключение с помощью raise
        raise NotImplementedError("Необходимо переопределить метод")
class Class2(Class1): # Наследуем абстрактный метод

```

```

def func(self, x): # Переопределяем метод
    print(x)
class Class3(Class1): # Класс не переопределяет метод
    pass
c2 = Class2()
c2.func(50) # Выведет: 50
c3 = Class3()
try: # Перехватываем исключения
    c3.func(50) # Ошибка. Метод func() не переопределен
except NotImplementedError as msg:
    print(msg) # Выведет: Необходимо переопределить метод

```

Результат работы приложения:

50

Необходимо переопределить метод

В состав стандартной библиотеки входит модуль **abc**. В этом модуле определен декоратор **@abstractmethod**, который позволяет указать, что метод, перед которым расположен декоратор, является абстрактным. При попытке создать экземпляр класса-потомка, в котором не переопределен абстрактный метод, возбуждается исключение **TypeError**. Рассмотрим использование декоратора **@abstractmethod** на примере:

```

from abc import ABCMeta, abstractmethod
class Class1(metaclass=ABCMeta):
    @abstractmethod
    def func(self, x): # Абстрактный метод
        pass
class Class2(Class1): # Наследуем абстрактный метод
    def func(self, x): # Переопределяем метод
        print(x)
class Class3(Class1): # Класс не переопределяет метод
    pass
c2 = Class2()
c2.func(50) # Выведет: 50
try:
    c3 = Class3() # Ошибка. Метод func не переопределен
    c3.func(50)
except TypeError as msg:
    print(msg) # Can't instantiate abstract class Class3
               # with abstract methods func

```

Результат работы приложения:

50

Can't instantiate abstract class Class3 with abstract methods func

Имеется возможность создания абстрактных статических методов и абстрактных методов класса, для чего необходимые декораторы указываются одновременно, друг за другом. Для примера объявим класс с абстрактными статическим методом и методом класса:

```

from abc import ABCMeta, abstractmethod
class MyClass(metaclass=ABCMeta):
    @staticmethod
    @abstractmethod
    def static_func(self, x):  # Абстрактный статический метод
        pass
    @classmethod
    @abstractmethod
    def class_func(self, x):  # Абстрактный метод класса
        pass

```

4.5 Ограничение доступа к идентификаторам внутри класса

Все идентификаторы внутри класса в языке **Python** являются открытыми, т. е. доступны для непосредственного изменения. Для имитации частных идентификаторов можно воспользоваться методами `__getattr__()`, `__getattribute__()` и `__setattr__()`, которые перехватывают обращения к атрибутам класса. Кроме того, можно воспользоваться идентификаторами, названия которых начинаются с двух символов подчеркивания. Такие идентификаторы называются **псевдочастными**. Псевдочастные идентификаторы доступны лишь внутри класса, но никак не вне его. Тем не менее, изменить идентификатор через экземпляр класса все равно можно, зная, каким образом искажается название идентификатора. Например, идентификатор `__privateVar` внутри класса **Class1** будет доступен по имени `_Class1__privateVar`. Как можно видеть, здесь перед идентификатором добавляется название класса с предваряющим символом подчеркивания. Приведем пример использования псевдочастных идентификаторов:

```

class MyClass:
    def __init__(self, x):
        self.__privateVar = x
    def set_var(self, x):  # Изменение значения
        self.__privateVar = x
    def get_var(self):  # Получение значения
        return self.__privateVar
c = MyClass(10)  # Создаем экземпляр класса
print(c.get_var())  # Выведет: 10
c.set_var(20)  # Изменяем значение
print(c.get_var())  # Выведет: 20
try:  # Перехватываем ошибки
    print(c.__privateVar)  # Ошибка!!!
except AttributeError as msg:
    print(msg)  # Выведет: 'MyClass1 object has
                # no attribute '__privateVar'
c._MyClass__privateVar = 50  # Значение псевдочастных атрибутов
                             # все равно можно изменить
print(c.get_var())  # Выведет: 50

```

Результат работы приложения:

```
10
20
'MyClass' object has no attribute '__privateVar'
50
```

Можно также ограничить перечень атрибутов, разрешенных для экземпляров класса. Для этого разрешенные атрибуты перечисляются внутри класса в атрибуте `__slots__`. В качестве значения атрибуту можно присвоить строку или список строк с названиями идентификаторов. Если производится попытка обращения к атрибуту, не перечисленному в `__slots__`, то возбуждается исключение **AttributeError**:

```
class MyClass:
    __slots__ = ["x", "y"]
    def __init__(self, a, b):
        self.x, self.y = a, b
c = MyClass(1, 2)
print(c.x, c.y) # Выведет: 1 2
c.x, c.y = 10, 20 # Изменяем значения атрибутов
print(c.x, c.y) # Выведет: 10 20
try: # Перехватываем исключения
    c.z = 50 # Атрибут z не указан в __slots__,
            # поэтому возбуждается исключение
except AttributeError as msg:
    print(msg) # 'MyClass' object has no attribute 'z'
```

Результат работы приложения:

```
1 2
10 20
'MyClass' object has no attribute 'z'
```

4.6 Инкапсуляция

Скрытие внутреннего устройства объектов называется **инкапсуляцией** и является одним из основных принципов ООП. Такой подход позволяет обезопасить внутренние данные (поля) объекта от изменений (возможно, разрушительных) со стороны других объектов; проверять корректность данных, поступающих от других объектов, повышая тем самым надежность программного кода; передавать внутреннюю структуру и код объекта любым способом, не меняя его внешние характеристики (интерфейс), и при этом никакой переделки других объектов не требуется.

По умолчанию все атрибуты и методы объекта являются **открытыми**. Соответственно, из основного кода программы можно использовать значения атрибутов и вызывать методы. Но для реализации инкапсуляции можно сделать атрибуты и методы класса **закрытыми**.

Предположим, при создании класса **Student** мы хотим оставить открытыми атрибуты, отвечающие за имя студента и город, в котором он проживает.

Информацию о возрасте студента мы хотим сделать недоступной. Для того чтобы сделать подобный атрибут закрытым, мы начнем запись его в программе с двух нижних подчеркиваний, например, **self.__vozs**. В следующем коде показано, что доступ к закрытому атрибуту внутри объявления класса может быть осуществлен очень просто. Результатом выполнения программы станет вывод сообщения: «Здравствуйте, я – студент Сергей из Москвы 17».

```
class Student:
    def __init__(self, name, city, vozs):
        self.name=name #Открытые атрибуты
        self.city=city
        self.__vozs=vozs #Закрытый атрибут
    def message(self):
        print("Здравствуйте, я-студент", self.name, "из", self.city, self.__vozs)
st1=Student("Сергей", "Москвы", "17")
st1.message()
```

Продемонстрировать ситуацию, когда закрытие атрибута приводит к ограничению доступа к нему из основного кода, можно на следующем примере. Попытка обратиться к атрибуту **vozs** вне объявления класса **Student** (т. е. просто из командной строки) приводит к ошибке, в которой сообщается о том, что объект **Student** не имеет атрибута **vozs** (рис. 159). Такая же ошибочная ситуация возникнет даже в том случае, если указать атрибут с двумя нижними подчеркиваниями, например, **print(st1.__vozs)**.

Итак, рассказав о методах создания закрытых атрибутов и способах доступа к ним, можно перейти к разговору о создании **закрытых методов**. Для этого дополним код методом **__talk()**, с помощью которого выведем сообщение: "Начнем урок".

```
def __talk(self):
    print("Начнем урок")
```

Вызовем созданный метод из метода **message()**, созданного ранее, оператором **self.__talk()**.

```
class Student:
    def __init__(self, name, city, vozs):
        self.name=name #Открытые атрибуты
        self.city=city
        self.__vozs=vozs #Закрытый атрибут
    def __talk(self): #Закрытый метод
        print("Начнем урок")
    def message(self):
        print("Здравствуйте, я - студент", self.name, "из", self.city, self.__vozs)
        self.__talk()
st1 = Student("Сергей", "Москвы", "17")
st1.message()
```

После создания закрытого метода можно продемонстрировать ситуацию, когда из пользовательского кода доступ к такому методу не может быть осуществлен. В частности, обращение к классу **Student** с указанием метода

`__talk()` вызовет исключение **AttributeError** с сообщением об отсутствии соответствующего атрибута.

Тем не менее, существует возможность вызвать закрытый метод (или получить значение скрытого атрибута), обратившись к имени класса с помощью следующего синтаксиса:

`__ИмяКласса__ИмяАтрибута`

Давайте получим доступ к закрытому атрибуту в нашей программе, написав оператор `print(st1._Student__vozr)`, и вызовем закрытый метод, оператором `st1._Student__talk()`.

После работы над созданием закрытых атрибутов и методов может возникнуть естественный вопрос: «А зачем их делать закрытыми, если доступ к ним все равно имеется?» Ответ можно дать такой: во-первых, случайно воспользоваться закрытым атрибутом или методом не удастся, а, во-вторых, инкапсуляция, т. е. ограничение доступа к методам или переменным, в Python существует на уровне договоренности между программистами, поэтому не стоит стремиться закрывать абсолютно все методы и атрибуты созданного класса.

4.7 Пример решения задачи на языке программирования Python

Задача. Создайте класс Time (Время), который предназначен для хранения времени в минутах. В этом классе реализуйте перегрузку операторов сложения и вычитания так, чтобы можно было выполнять операции между двумя объектами времени. При сложении двух объектов времени их значения должны суммироваться, а при вычитании — находиться разность между ними. Результат выполнения операций должен выводиться в виде общего количества минут.

Решение задачи:

```
class Time:
    def __init__(self, minutes):
        self.minutes = minutes
    def __add__(self, other):
        return Time(self.minutes + other.minutes)
    def __sub__(self, other):
        return Time(self.minutes - other.minutes)
    def __str__(self):
        return f"{self.minutes} минут"

# Пример использования
t1 = Time(30)
t2 = Time(20)
print("t1 =", t1)
print("t2 =", t2)
print("Сложение:", t1 + t2)
print("Вычитание:", t1 - t2)
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Опишите класс `Rectangle` (прямоугольник), в котором имеется статический метод, вычисляющий площадь прямоугольника, принимающий два параметра (ширину и длину). Метод вызывается в конструкторе для вычисления площади конкретного прямоугольника и результат присваивается локальному свойству создаваемого экземпляра класса и реализуйте возможность создания не более трех экземпляров этого класса.

2 Опишите класс `Dog` (собака), в котором имеется статический метод, вычисляющий год рождения собаки, и примет один параметр (возраст собаки). В каждом его экземпляре создавайте несколько локальных свойств (например: имя, возраст, порода) и реализуйте возможность создания не более пяти экземпляров этого класса.

3 Опишите класс `Calendar` для хранения даты, в котором имеется статический метод, вычисляющий количество лет, прошедших с начала 20 века и примет один параметр (номер текущего года). В каждом его экземпляре создавайте несколько локальных свойств (например: день, месяц, год) и реализуйте возможность создания не более четырех экземпляров этого класса.

4 Опишите класс `Circle` (окружность), в котором имеется статический метод, вычисляющий площадь окружности, принимающий радиус окружности в качестве параметра. В каждом экземпляре класса создайте локальное свойство для хранения радиуса и реализуйте возможность создания не более десяти экземпляров этого класса.

5 Опишите класс `Student` (студент), в котором имеется статический метод, вычисляющий средний балл студента, принимающий список оценок в качестве параметра. В каждом экземпляре класса создайте локальные свойства для хранения имени студента и списка его оценок. Реализуйте возможность создания не более пятидесяти экземпляров этого класса.

6 Опишите класс `Car` (автомобиль), в котором имеется статический метод, вычисляющий средний расход топлива на 100 км, принимающий объем бензобака и пройденное расстояние в качестве параметров. В каждом экземпляре класса создайте локальные свойства для хранения марки автомобиля, объема бензобака и пройденного расстояния. Реализуйте возможность создания не более двадцати экземпляров этого класса.

7 Опишите класс `Book` (книга), в котором имеется статический метод, вычисляющий среднюю оценку книги на основе рейтингов пользователей, принимающий список оценок в качестве параметра. В каждом экземпляре класса создайте локальные свойства для хранения названия книги, автора и списка оценок. Реализуйте возможность создания не более ста экземпляров этого класса.

8 Опишите класс `Triangle` (треугольник), в котором имеется статический метод, вычисляющий площадь треугольника по формуле Герона, принимающий

длины трех сторон в качестве параметров. В каждом экземпляре класса создайте локальные свойства для хранения длин сторон треугольника. Реализуйте возможность создания не более десяти экземпляров этого класса.

9 Опишите класс `Employee` (сотрудник), в котором имеется статический метод, вычисляющий среднюю заработную плату сотрудника за месяц на основе его оклада и количества отработанных часов, принимающий оклад и количество отработанных часов в качестве параметров. В каждом экземпляре класса создайте локальные свойства для хранения имени сотрудника, его должности, оклада и количества отработанных часов. Реализуйте возможность создания не более двадцати экземпляров этого класса.

10 Опишите класс `Product` (продукт), в котором имеется статический метод, вычисляющий общую стоимость продукта на основе его цены и количества, принимающий цену и количество в качестве параметров. В каждом экземпляре класса создайте локальные свойства для хранения названия продукта, его цены и количества. Реализуйте возможность создания не более ста экземпляров этого класса.

11 Опишите класс `Car` (автомобиль), в котором имеется статический метод для вычисления средней скорости движения автомобиля на основе пройденного расстояния и времени, принимающий расстояние и время в качестве параметров. В каждом экземпляре класса создайте локальные свойства для хранения марки автомобиля, его модели, года выпуска и пробега. Реализуйте возможность создания не более ста экземпляров этого класса.

12 Опишите класс `Circle` (окружность), в котором имеется статический метод для вычисления длины окружности и площади круга на основе его радиуса, принимающий радиус в качестве параметра. В каждом экземпляре класса создайте локальное свойство для хранения радиуса окружности. Реализуйте возможность создания не более десяти экземпляров этого класса.

13 Опишите класс `Movie` (фильм), в котором имеется статический метод для вычисления среднего возраста зрителей, просмотревших фильм, на основе возрастов зрителей, принимающий список возрастов в качестве параметра. В каждом экземпляре класса создайте локальные свойства для хранения названия фильма, режиссера и списка возрастов зрителей. Реализуйте возможность создания не более ста экземпляров этого класса.

14 Опишите класс `Laptop` (ноутбук), в котором имеется статический метод, вычисляющий среднее время автономной работы устройства на основе списка значений времени работы в часах (принимающий список значений в качестве параметра). В каждом экземпляре класса создайте локальные свойства для хранения модели ноутбука, производителя, объёма оперативной памяти и ёмкости аккумулятора. Реализуйте возможность создания не более пятидесяти экземпляров этого класса.

15 Опишите класс `BankAccount` (банковский счёт), в котором имеется статический метод, вычисляющий итоговый баланс после начисления процентов, принимающий исходную сумму и процентную ставку в качестве параметров. В каждом экземпляре класса создайте локальные свойства для хранения имени

владельца счёта, номера счёта и текущего баланса. Реализуйте возможность создания не более ста экземпляров этого класса.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте класс «Вектор», который будет представлять собой вектор в трехмерном пространстве. Перегрузите оператор сложения для векторов.

2 Создайте класс «Матрица», который будет представлять собой квадратную матрицу. Перегрузите оператор умножения для матриц.

3 Создайте класс «Комплексное число», который будет представлять собой комплексное число. Перегрузите оператор умножения для комплексных чисел.

4 Создайте класс «Студент», содержащий информацию о имени, возрасте и среднем балле студента. Перегрузите оператор сравнения для сравнения средних баллов студентов.

5 Создайте класс «Время», представляющий время в часах и минутах. Перегрузите оператор сложения для добавления времени.

6 Создайте класс «Деньги», представляющий сумму денег. Перегрузите операторы сложения и вычитания для операций с деньгами.

7 Создайте класс «Книга», содержащий информацию о названии и количестве страниц. Перегрузите оператор сравнения для сравнения количества страниц книг.

8 Создайте класс «Треугольник», представляющий геометрический треугольник. Перегрузите операторы сложения и вычитания для операций с площадями треугольников.

9 Создайте класс «Полином», представляющий многочлен. Перегрузите оператор умножения для умножения полиномов.

10 Создайте класс «Товар», содержащий информацию о названии товара и его цене. Перегрузите оператор сравнения для сравнения цен товаров.

11 Создайте класс «Очередь», представляющий структуру данных очередь. Перегрузите оператор добавления элемента в очередь.

12 Создайте класс «Матрица смежности», представляющий граф. Перегрузите операторы сложения и вычитания для операций объединения и разности графов.

13 Создайте класс «Комплексная функция», представляющий функцию комплексной переменной. Перегрузите оператор умножения для композиции функций.

14 Создайте класс «Координаты точки», который представляет точку на плоскости (x, y). Перегрузите оператор сложения для перемещения точки на заданный вектор и оператор вычитания для нахождения вектора между двумя точками.

15 Создайте класс «Интервал», который представляет числовой интервал (начало и конец). Перегрузите оператор сложения для объединения интервалов и оператор сравнения для проверки, какой интервал больше по длине.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Перечислите и опишите специальные методы в языке программирования Python.

7.2 Перечислите и опишите методы для перегрузки математических операторов в языке программирования Python.

7.3 Охарактеризуйте статические методы и методы класса в языке программирования Python.

7.4 Охарактеризуйте абстрактные методы в языке программирования Python.

7.5 Объясните роль статических методов в языке программирования Python.

7.6 Перечислите и опишите методы перегрузки двоичных операторов в языке программирования Python.

Список используемых источников

1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.